

HEARTSENSE

Heart Disease Prediction Project Code Guide

A concise explanation of the important Python, Django, machine-learning, database, authentication, interface, and PDF-report code.

The one-sentence explanation

HeartSense is a Django web application that accepts 13 clinical inputs, standardizes them, passes them to a saved logistic-regression model, stores the result per signed-in user, and lets the user download a PDF report.

What to emphasize to your lecturer

- The ML model is trained separately, then loaded by the web application for prediction.
- Authentication and ownership checks keep normal users limited to their own records.
- Django connects the HTML interface, prediction logic, SQLite database, and report download.
- The output is an educational screening result, not a medical diagnosis.

Prepared from the code in `heart-disease-prediction-main`.

Presentation date: 9 September 2026

1. System overview

The application has two phases. During **training**, a CSV dataset is split, standardized, and used to fit logistic regression. During **web use**, Django loads the saved scaler and model, processes a form submission, displays the classification, and saves a short record.



Main technologies and their jobs

Technology	Purpose in this project	Main evidence
Django	Web routing, views, templates, sessions, authentication, messages, ORM	settings.py, urls.py, views.py
Scikit-learn	StandardScaler preprocessing and LogisticRegression classification	ml/train_model.py
Pandas	Loads heart.csv and separates inputs from target	ml/train_model.py
Joblib	Saves and reloads the trained model and scaler	train_model.py, views.py
SQLite	Stores users and prediction history	settings.py, models.py
ReportLab	Builds a downloadable patient-result PDF in memory	predictor/pdf.py
HTML/CSS/Bootstrap	Responsive dashboard, forms, history table, alerts	templates/, static/css/style.css

Important directory map

Path	What it contains
heart_project/	Project-level configuration and root URL routing.
predictor/	The main Django app: views, model, forms, URLs, templates, ML artifacts, PDF logic.
ml/train_model.py	Offline training and evaluation script.
dataset/heart.csv	303 rows, 13 input columns, and the target label.
static/css/style.css	Visual design and responsive styling.

Useful distinction

The **Patient** class is a database model. The **heart_model.pkl** file is the machine-learning model. They are different kinds of 'model'.

2. Machine-learning training

File: **ml/train_model.py**. This code creates the two artifacts used at runtime: **heart_model.pkl** and **scaler.pkl**.

Key code: prepare data, preserve class proportions, standardize values, and fit the classifier.

```
df = pd.read_csv("dataset/heart.csv")
X = df.drop("target", axis=1)
y = df["target"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)
```

- **X** contains the 13 predictors; **y** is the target (0 or 1).
- **test_size=0.2** holds out about 20% of the dataset for evaluation.
- **random_state=42** makes the split reproducible.
- **stratify=y** keeps the class balance similar in training and test data.
- The scaler is fitted only on training data, preventing test-data leakage.
- Logistic regression estimates the probability of the two classes and applies a decision boundary.

Verified model facts

Item	Verified value	Meaning
Dataset	303 rows; 13 features + target	A small educational dataset.
Target balance	138 class 0; 165 class 1	Slightly more positive samples.
Saved classifier	LogisticRegression	Binary linear probabilistic classifier.
Saved preprocessor	StandardScaler	Centers/scales every input feature.
Accuracy	80.33%	49 of 61 test cases classified correctly.
Precision	76.92%	Of predicted positives, about 77% were positive.
Recall	90.91%	Detected 30 of 33 positives in this test split.
F1 score	83.33%	Balance of precision and recall.
Confusion matrix	[[19, 9], [3, 30]]	19 TN, 9 FP, 3 FN, 30 TP.

How to present the metric

Say: 'On the reproducible 20% holdout split, the saved model achieved 80.33% accuracy and 90.91% recall.' Do not claim these numbers prove clinical safety.

3. The core prediction view

File: **predictor/views.py**. The **home()** function is the central controller. The login decorator prevents anonymous access, then POST requests are parsed and converted into exactly the feature order used during training.

Key code: form submission -> ordered numeric row -> scaling -> classification and confidence.

```
@login_required
def home(request):
    if request.method == "POST":
        age = float(request.POST["age"])
        sex = int(request.POST["sex"])
        # ...read the remaining 11 values...

        data = [[age, sex, cp, trestbps, chol, fbs,
                  restecg, thalach, exang, oldpeak,
                  slope, ca, thal]]

        scaled_data = scaler.transform(data)
        prediction = model.predict(scaled_data)[0]
        probability = model.predict_proba(scaled_data)
        confidence = round(max(probability[0]) * 100, 2)
```

Why each line matters

Code idea	Purpose	Presentation note
@login_required	Redirects anonymous visitors to login.	The prediction page is account-protected.
request.POST[...]	Gets submitted HTML form values.	Conversion to int/float is required for ML.
data = [...]	Creates one sample with 13 features.	Order must match heart.csv training columns.
scaler.transform	Applies training-time scaling without refitting.	Never use fit_transform on a new patient.
model.predict	Returns class 0 or 1.	1 maps to disease detected.
predict_proba	Returns probability for each model class.	The maximum is shown as model confidence.

Key code: translate the numeric class into readable text and save the record through Django ORM.

```
result = (
    "Heart Disease Detected"
    if prediction == 1
    else "No Heart Disease Detected"
)

Patient.objects.create(
    user=request.user,
    name=name, email=email, phone=phone,
    age=int(age), gender=gender,
    prediction=result, confidence=confidence,
)
```

Confidence is easy to misdescribe

The displayed percentage is the model's highest predicted class probability. It is not a medically calibrated risk score and it is not proof that the answer is correct.

4. Authentication and access control

The project uses Django's built-in **User**, **AuthenticationForm**, **UserCreationForm**, sessions, password validators, and CSRF protection. Custom code adds friendly fields, messages, safe redirects, and record ownership.

Login code: authenticate, create a session, and allow only a safe local next-page redirect.

```
form = LoginForm(request=request, data=request.POST or None)
if request.method == "POST" and form.is_valid():
    user = form.get_user()
    login(request, user)

    next_url = request.POST.get("next") or request.GET.get("next")
    if next_url and url_has_allowed_host_and_scheme(
        url=next_url,
        allowed_hosts={request.get_host()},
        require_https=request.is_secure(),
    ):
        return redirect(next_url)
```

Authorization code: staff can see all records; a normal user receives only their own queryset.

```
@login_required
def history(request):
    if request.user.is_staff:
        patients = Patient.objects.all().order_by("-created_at")
    else:
        patients = Patient.objects.filter(
            user=request.user
        ).order_by("-created_at")
```

Security decisions already present

- Prediction, history, and PDF download views require a logged-in user.
- The PDF lookup includes **user=request.user** for normal accounts, preventing simple ID-based access to another user's report.
- Logout accepts POST only through **@require_POST**.
- Templates include **{% csrf_token %}** in POST forms.
- Signup checks email uniqueness without case sensitivity.
- Django's configured password validators reject weak/common passwords.

Good lecturer answer

Authentication answers 'Who are you?' Authorization answers 'Which records may you access?' This project implements both.

5. Database, routes, and Django structure

The **Patient** model defines the stored prediction summary. Django migrations create and evolve its SQLite table. Each record may point to a Django user through a foreign key.

Database model: fields become columns; `Patient.objects.create(...)` inserts a row.

```
class Patient(models.Model):
    user = models.ForeignKey(
        settings.AUTH_USER_MODEL,
        on_delete=models.SET_NULL,
        null=True, blank=True,
        related_name="predictions",
    )
    name = models.CharField(max_length=100)
    email = models.EmailField()
    phone = models.CharField(max_length=15)
    age = models.IntegerField()
    gender = models.CharField(max_length=10)
    prediction = models.CharField(max_length=50)
    confidence = models.FloatField()
    created_at = models.DateTimeField(auto_now_add=True)
```

- **SET_NULL** preserves a prediction if its linked user is deleted, but removes the owner link.
- **related_name='predictions'** allows access such as `user.predictions.all()`.
- **auto_now_add=True** automatically records creation time.
- Admin configuration adds useful columns, filters, search fields, and a read-only timestamp.

URL map

URL	View	Purpose
/	home	Dashboard and prediction form.
/login/	login_view	Authenticate an existing user.
/signup/	signup_view	Create and sign in a new user.
/logout/	logout_view	End session; POST only.
/history/	history	Show account-filtered saved predictions.
/download/<id>/	download_pdf	Generate one authorized report.
/admin/	Django admin	Staff administration interface.

Settings that connect the application

- **'predictor'** is registered in `INSTALLED_APPS`.
- The default database engine is SQLite at `db.sqlite3`.
- Project URLs include **predictor.urls** at the site root.
- The time zone is **Asia/Kuala_Lumpur**.
- Static CSS is served from the project **static/** directory in development.

6. Interface, history, and PDF reporting

Django templates render server data using `{{ variables }}` and control flow using `{% tags %}`. `base.html` supplies the navigation, messages, footer, Bootstrap, Font Awesome, and shared CSS; child templates fill its content blocks.

Template logic: show the latest result only when one exists, with risk/clear styling.

```
{% if result %}
<section class="result-card
  {% if result == 'Heart Disease Detected' %}risk
  {% else %}clear{% endif %}">
  <h2>{{ result }}</h2>
  <strong>{{ confidence }}</strong>
</section>
{% endif %}
```

The dashboard also receives per-user summary counts from the view: total predictions, detected cases, and non-detected cases. The history template loops over the authorized queryset and creates one table row per prediction.

Report code: ReportLab builds the PDF in memory; the view returns those bytes as `application/pdf`.

```
def generate_pdf(patient):
    buffer = BytesIO()
    doc = SimpleDocTemplate(buffer)
    styles = getSampleStyleSheet()
    story = [
        Paragraph("<b>Heart Disease Prediction Report</b>",
            styles["Title"]),
        Paragraph(f"<b>Name:</b> {patient.name}",
            styles["BodyText"]),
        # ...other stored fields...
    ]
    doc.build(story)
    return buffer.getvalue()
```

Download response: content type identifies a PDF; Content-Disposition requests a file download.

```
pdf = generate_pdf(patient)
response = HttpResponse(pdf, content_type="application/pdf")
response["Content-Disposition"] = (
    f'attachment; filename="Patient_{patient.id}.pdf"'
)
return response
```

Why BytesIO?

It avoids creating a temporary report file on the server. The PDF exists in memory just long enough to be returned to the browser.

7. The 13 model inputs

The feature order below is critical. The runtime list in **views.py** matches the dataset columns used by the training script. Changing the order silently changes what the model thinks each number means.

Column	Meaning	Value used by the app
age	Age	Years
sex	Sex	0 female, 1 male in this UI
cp	Chest pain type	Encoded category 0-3
trestbps	Resting blood pressure	mmHg
chol	Serum cholesterol	mg/dL
fb	Fasting blood sugar > 120	0 no, 1 yes
restecg	Resting ECG	Encoded category 0-2
thalach	Maximum heart rate	beats per minute
exang	Exercise-induced angina	0 no, 1 yes
oldpeak	ST depression	Numeric value
slope	Peak exercise ST slope	Encoded category 0-2
ca	Major vessels	Count/category 0-4
thal	Thalassemia test category	Encoded category 0-3

Important limitation

The database record stores patient contact details, age, gender, result, and confidence, but it does **not** store the other 11 clinical inputs or a model version. Therefore, an old prediction cannot be fully reproduced or audited later.

Target interpretation

- Class **1** becomes 'Heart Disease Detected'.
- Class **0** becomes 'No Heart Disease Detected'.
- The wording is stronger than the evidence supports; 'higher/lower predicted risk' would be safer for a screening demo.

8. Critical review and recommended improvements

A strong presentation includes both what works and what should be improved. These findings come directly from the supplied code.

Priority	Area	Current issue	Recommended change
High	Production configuration	SECRET_KEY is in source, DEBUG=True, and ALLOWED_HOSTS is empty.	Move secrets to environment variables; configure production settings.
High	Clinical interpretation	Output says disease is detected and confidence looks authoritative.	Use risk wording, validate/calibrate the model, and retain the disclaimer.
High	Personal data	Name, email, phone, and health result are stored in SQLite.	Minimize data, define retention, restrict admin, encrypt/back up appropriately.
Medium	Server validation	Clinical values are parsed directly from request.POST; ranges mainly live in HTML.	Create a Django PredictionForm with server-side ranges and category choices.
Medium	Auditability	Most clinical inputs and model version are not saved.	Store all inputs, artifact/version ID, and an immutable prediction timestamp.
Medium	Model evaluation	303 rows and one train/test split; no cross-validation or external validation.	Add cross-validation, ROC-AUC, calibration, subgroup checks, and external data.
Medium	Automated tests	predictor/tests.py contains no tests.	Test login, ownership, invalid form values, model ordering, history, and PDF access.
Low	Training cleanup	heart_model.pkl is dumped twice consecutively.	Remove the duplicate joblib.dump call.
Low	Error handling	A broad Exception hides the underlying failure.	Log the exception server-side and show a safe user message.
Low	PDF text safety	User text is inserted into ReportLab Paragraph markup without escaping.	Escape user values before building Paragraph content.

Balanced conclusion

The application is a clear end-to-end academic prototype: it successfully connects data preparation, ML inference, Django authentication, persistence, and reporting. Its main next step is stronger validation and production-grade privacy/security.

9. Five-minute presentation script

Time	Suggested wording
0:00 - Problem	'The goal is to demonstrate how machine learning can be integrated into a web application to provide an educational heart-risk screening result from 13 clinical inputs.'
0:40 - Architecture	'Django manages pages, authentication, database access, and downloads. A separately trained scikit-learn model handles classification. SQLite stores each user's prediction summary.'
1:20 - Training	'The CSV is split 80/20 with stratification. StandardScaler is fitted on training data only, and logistic regression is trained. Both objects are saved with joblib.'
2:10 - Runtime	'When a signed-in user submits the form, the view converts the 13 values, keeps the training column order, applies the saved scaler, calls predict and predict_proba, then renders and stores the result.'
3:05 - Security	'Django protects the main views. Normal users query and download only records linked to their account, while staff can access all records.'
3:50 - Results	'On the reproducible holdout split, the saved model gives 80.33% accuracy and 90.91% recall. This is coursework evidence, not clinical validation.'
4:25 - Reflection	'The strongest improvement would be server-side clinical validation, complete audit records, automated tests, secure production settings, and broader model validation.'

Likely lecturer questions

Question	Short answer
Why scale the input?	Features use different numeric ranges. Standardization helps logistic regression optimize and keeps runtime data consistent with training.
Why save the scaler too?	The exact training transformation must be reused. A newly fitted scaler would change the input representation.
Why logistic regression?	It is fast, interpretable, suitable for binary classification, and provides class probabilities.
How is user privacy enforced?	Queries and downloads filter Patient records by request.user; staff is the explicit exception.
Is confidence the chance the patient has disease?	No. It is the model's class probability for its chosen label and is not necessarily medically calibrated.
What would you improve first?	Use a validated Django form, store all inputs/model version, add tests, and harden settings and privacy controls.

10. Final presentation checklist

Use this page immediately before presenting.

Demo sequence

- Start the server and open **http://127.0.0.1:8000/**.
- Create an account or log in.
- Show that the dashboard requires authentication.
- Enter one complete set of clinical values and run the prediction.
- Point out the predicted class, model confidence, disclaimer, and updated statistics.
- Open History and download the PDF for that record.
- If possible, use a second account to demonstrate separated histories.

Commands

Normal Windows command-line startup. The project also provides RUN_PROJECT.cmd.

```
cd heart-disease-prediction-main
.venv\Scripts\activate
python manage.py migrate
python manage.py runserver
```

Do not overclaim

- Say **prototype**, **educational screening**, and **holdout result**.
- Do not say the tool diagnoses a patient or is hospital-ready.
- Explain that model performance depends on dataset quality and representativeness.
- Treat health and contact information as sensitive data.

Best final sentence

'This project demonstrates a complete ML web pipeline, and the code review shows a clear path from academic prototype to a safer, more testable system.'

Source files reviewed

README.md; ml/train_model.py; predictor/views.py; forms.py; models.py; pdf.py; urls.py; admin.py; migrations; templates; heart_project/settings.py; heart_project/urls.py; dataset/heart.csv; saved model and scaler metadata.